

FlashAttention-3：利用异步与低精度实现的快速且精确的注意力机制

Jay Shah^{*1}, Ganesh Bikshandj^{*1}, Ying Zhang², Vijay Thakkar^{3,4}, Pradeep Ramani³, Tri Dao^{5,6}

¹ Colfax Research, ² Meta, ³ NVIDIA (英伟达), ⁴ Georgia Tech (乔治亚理工学院), ⁵ Princeton University (普林斯顿大学), ⁶ Together AI

{jayshah, ganesh}@colfax-intl.com, yingz@meta.com, {vithakkar, prraman}@nvidia.com, tri@tridao.me
2024年7月16日

* 共同贡献

摘要

注意力机制作为无处不在的Transformer架构中的核心层，是大型语言模型和长上下文应用的性能瓶颈。FlashAttention通过将所有注意力操作融合到单个GPU内核中并最大限度减少对全局内存的读写，提出了一种在GPU上加速注意力的方法。然而，它尚未利用最新硬件的新能力：FlashAttention-2在H100 GPU上仅达到35%的利用率。我们针对Hopper架构GPU开发了三项主要技术来加速注意力计算：(1) 利用张量核心与TMA的异步执行，通过**Warp**级别的特化设计实现计算与数据传输的重叠；(2) 交错执行分块矩阵乘法（GEMM）与softmax操作，以在异步条件下隐藏softmax的开销；(3) 利用硬件对FP8低精度的支持，引入分块量化和不相干处理，以减少低精度带来的精度损失。

我们的方法FlashAttention-3在H100 GPU上的FP16精度实现相比FlashAttention-2实现了1.5–2.0倍的加速，前向计算吞吐量最高可达740 TFLOP/s（75%硬件利用率），FP8精度实现的吞吐量接近1.2 PFLOP/s。【^{^1}】我们验证了FP8精度的FlashAttention-3相较于基线FP8注意力计算的数值误差降低了2.6倍。此外，FP16精度的FlashAttention-3在保持与FlashAttention-2相当精度的同时（因为中间结果如softmax缩放仍以FP32保存），数值误差比标准的注意力实现更低。而结合分块量化和不相干处理的FP8精度FlashAttention-3比采用逐张量量化的标准FP8注意力精度提高了2.6倍。

1 引言

对于Transformer架构【59】而言，注意力机制的计算是主要的计算瓶颈，因为对 q 和 k 的自注意力分数计算在序列长度上是二次复杂度。将注意力扩展到更长上下文能够解锁模型的新能力（例如对多个长文档的建模与推理【24, 43, 50】，以及大型代码库中的文件【30, 48】）、新模态（高分辨率图像【11】、音频【23】、视频【25】）和新应用（具有长历史的用户交互【53】、长时间跨度的智能体工作流【62】）。因此，使注意力在长上下文场景下运行得更快成为了研究热点，包括通过近似方法【14, 27, 56】和软件优化【17, 29, 45】来加速，甚至提出替代架构【22, 42, 55】。

在本工作中，我们基于Dao等人【17】的研究，开发精确注意力算法，将对GPU执行模型和硬件特性的了解融入其高级设计中。在【17】中，Dao等人提出了FlashAttention，这是一种将注意力操作铺排的新颖算法，通过将所有注意力操作融合到单个GPU内核中，避免中间结果读写到缓慢的全局内存。随后，Dao【15】将该算法重构为

FlashAttention-2, 使其也能在序列长度维度上并行化, 并对前向传播的内环按 K 和 V 矩阵的块进行计算, 从而提升GPU上的占用率和工作分配均衡。然而, 我们观察到, 相比经过优化的矩阵乘法 (GEMM) 内核, 在新一代GPU上FlashAttention-2的利用率仍然很低, 例如在NVIDIA Hopper H100 GPU上仅约35% (而优化GEMM可达80–90%)。部分原因可能在于实现层面的差异, 例如在针对张量核心进行计算时, 没有使用Hopper特定的指令来替换Ampere架构的指令。一些工作 (如ThunderKitten [52] 和cuDNN 9 [39]) 已经表明, 通过使用Hopper特定指令和基于划分的抽象, 可以加速注意力计算并简化实现。

更根本地说, FlashAttention-2的算法遵循一个简化的同步模型, 在设计上没有显式利用**异步执行**和**低数值精度**。硬件为了加速机器学习工作负载中的关键操作而引入了异步执行: 专用硬件单元分别执行矩阵乘法 (张量核心) 和内存加载 (张量内存加速器, Tensor Memory Accelerator, 简称TMA), 与执行逻辑、整数和浮点运算的其余CUDA核心分离。同时, 低精度数值 (如Hopper架构提供的FP8和未来Blackwell架构的FP4, 继承了2017年Pascal架构的FP16和2020年Ampere架构的BF16) 已被证明能够在相同功耗和芯片面积下将吞吐量提高两倍或四倍。我们将在第5.2.2节回顾Hopper架构在这些方面提供的能力。技术挑战在于, 需重新设计FlashAttention-2以利用这些硬件特性: 异步执行要求在矩阵乘法和softmax之间重叠计算, 即使二者存在先后依赖关系; 低精度则要求格外小心以最小化量化误差, 尤其是处理大型语言模型中出现的离群特征时 [20, 54]。

为此, 我们提出了FlashAttention-3。该方法综合了三项新思想以在新GPU架构上进一步提升性能: [^1]

1. **生产者-消费者异步执行**: 我们定义了一种基于Warp特化的软件流水线方案, 它将数据生产者和消费者划分到不同的warp中, 利用张量内存加速器和张量核心的异步执行能力, 从而拓展算法隐藏内存访问和指令发射延迟的能力。
2. **在异步块状GEMM下隐藏Softmax计算**: 我们将softmax所涉及的**非GEMM**操作 (如浮点乘法和指数计算) 的低吞吐量部分, 与GEMM的异步WGMMMA指令穿插重叠执行。为此, 我们重新设计了FlashAttention-2算法以避免softmax与GEMM之间的某些顺序依赖。例如, 在我们算法的两阶段版本中, 当softmax在分块的Scores矩阵上执行时, WGMMMA指令异步地计算下一个块的乘法。
3. **硬件加速的低精度GEMM**: 我们调整了前向传播算法, 使其能够利用FP8张量核心执行矩阵乘法, 这使实测TFLOP/s几乎提高一倍。这需要解决FP8 WGMMMA指令对操作数内存布局的特殊要求: 对于SMEM中的操作数, FP16 WGMMMA接受按 m/n 维度连续 (mn-major) 或按 k 维度连续 (k-major) 的任意格式, 但FP8 WGMMMA仅支持k-major格式。此外, 在注意力计算等需要将两个GEMM融合在同一内核中的场景, 不同FP32累加器和FP8操作数的布局要求存在冲突, 这成为调用连续FP8 WGMMMA的障碍。在我们的FP8算法设计中, 我们针对这些布局限制做出了一些调整, 详见5.3.3节, 并通过分块量化和不相干处理技术来缓解降为FP8精度带来的精度损失。

为了从实证上验证我们的方法, 我们在H100 SXM5 GPU [^1] 上对FlashAttention-3进行了基准测试, 覆盖了一系列参数配置。结果显示: (1) 在FP16精度下, 相比FlashAttention-2, 前向计算加速1.5–2.0倍 (吞吐量最高达到740 TFLOP/s), 反向计算加速1.5–1.75倍; (2) 在FP8精度下, 吞吐量接近1.2 PFLOP/s; (3) 在长序列长度下, FP16精度的FlashAttention-3性能优于cuDNN提供的最先进实现, FP8精度的FlashAttention-3性能也相当

[^2]。我们还验证了FP16精度的FlashAttention-3与FlashAttention-2具有相同级别的数值误差, 并且由于中间结果 (例如softmax缩放) 保留在FP32中, 其精度优于标准的注意力实现。此外, 结合了分块量化和不相干处理的FP8精度FlashAttention-3, 相比采用逐张量量化的标准FP8注意力在存在离群特征的情况下数值精度提升了2.6倍。

我们已在开源协议下发布了FlashAttention-3的源码 [^3]。我们计划将其集成入PyTorch和Hugging Face等深度学习库, 以惠及更多的研究人员和开发者。

[^1]: 我们在本文中以NVIDIA Hopper架构为背景描述结果。然而，只要GPU架构具备足够完善的异步执行和低精度支持，我们的方法同样适用。

[^3]: FlashAttention-3源码已发布在 GitHub：<https://github.com/Dao-AILab/flash-attention>。

2 背景：多头注意力机制与GPU特性

2.1 多头注意力机制

设 $Q, K, V \in \mathbb{R}^{N \times d}$ 分别表示单个注意力头的查询（query）、键（key）和值（value）序列，其中 N 是序列长度， d 是注意力头维度。那么注意力输出 O 计算如下：

$$S = \alpha Q K^T \in \mathbb{R}^{N \times N}, \quad P = \text{softmax}(S) \in \mathbb{R}^{N \times N}, \quad O = P V \in \mathbb{R}^{N \times d},$$

其中softmax在矩阵 S 的行上逐行计算， $\alpha = 1/\sqrt{d}$ 通常作为缩放因子。在实现中，我们会从 S 中减去每行的最大值 $\text{rowmax}(S)$ 以防止指数函数计算时产生数值不稳定。对于多头注意力

（MHA），每个注意力头都有自己的一组 Q, K, V 投影，多个头以及批次可以并行计算，拼接后得到完整的输出张量。

令 ϕ 为标量损失函数， $d(\cdot) = \partial \phi / \partial (\cdot)$ 表示梯度。给定输出梯度 $dO \in \mathbb{R}^{N \times d}$ ，我们根据链式法则计算 dQ, dK, dV 如下：

$$\begin{aligned} dV &= P^T dO \in \mathbb{R}^{N \times d}, \\ dP &= dO V^T \in \mathbb{R}^{N \times N}, \\ dS &= d\text{softmax}(dP) \in \mathbb{R}^{N \times N}, \\ dQ &= \alpha dS K \in \mathbb{R}^{N \times d}, \\ dK &= \alpha dS^T Q \in \mathbb{R}^{N \times d}. \end{aligned}$$

其中，用于向量 $\mathbf{s}$ 的softmax函数满足关系：若 $\mathbf{p} = \text{softmax}(\mathbf{s})$ ，则 $d\mathbf{s} = (\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^T) d\mathbf{p}$ 。我们将上述公式按行应用于矩阵情形，并用 $d\text{softmax}(dP)$ 表示这一逐行计算。最后，对于MHA的反向传播，上述计算也可以在多个注意力头和批次上并行执行。

2.2 GPU硬件特性与执行模型

我们首先描述与FlashAttention-3相关的GPU执行模型的一些方面，重点以NVIDIA Hopper架构作为具体实例。

内存层次结构：GPU的内存按层次组织，不同层级在容量和带宽上此消彼长（见表1【^4】）。全局内存

（GMEM，也称HBM）是片外DRAM，所有流式多处理器（SM）都可访问。来自GMEM的数据会被透明地缓存到片上L2缓存中。接下来，每个SM包含一个小型片上、由程序员管理且高度分银行的缓存——**共享内存**（SMEM）。最后，每个SM还有寄存器文件，每个线程最多可使用256个寄存器（称为**寄存器内存**，RMEM）。

表1：NVIDIA Hopper H100 SXM5 GPU的线程-内存层次结构。

硬件层级	并行单位	数据位置	容量 @ 带宽
芯片	网格 (Grid)	全局内存 (HBM)	80 GiB @ 3.35 TB/s
GPC	线程块簇	L2缓存	50 MiB @ 12 TB/s
SM	线程块 (CTA)	共享内存 (SMEM)	每个SM 228 KiB, 全GPU合计31 TB/s 带宽
线程	线程	寄存器 (RMEM)	每个SM 256 KiB

线程层次结构：GPU的编程模型围绕逻辑执行单元的层次结构组织。从最细粒度到最粗粒度，这个层次包括：线程 (thread)、**Warp** (由32个线程组成)、**Warp组** (warpgroup, 4个连续warp组成)、**线程块** (threadblock, 即CTA, 合作线程数组)、**线程块簇** (threadblock cluster, Hopper架构中新加入的层级) 和网格 (grid)。

上述两个层次结构紧密关联：同一CTA中的线程被调度到同一SM上，同一簇中的CTAs被调度到同一GPC上。共享内存 (SMEM) 可被CTA内的所有线程直接寻址访问，而每个线程有最多256个私有寄存器。

异步执行与Warp特化：GPU是**高吞吐处理器**，通过并发和异步执行来隐藏内存访问和指令执行的延迟。对于 GMEM与SMEM之间的异步内存拷贝操作，Hopper架构提供了专门的硬件单元**张量内存加速器** (Tensor Memory Accelerator, TMA) [38, §7.29]。此外，与先前的架构 (如Ampere) 不同，Hopper架构的张量核心通过Warp组范围的WGMMMA指令 [40, §9.7.14] 提供，对warp组可见，并且也支持异步执行，可以直接从共享内存读取其输入。硬件对异步执行的支持允许实现**Warp特化**的内核：将一个CTA内的不同warp分配为只执行数据搬运 (producer) 或只执行计算 (consumer) 的不同角色。通常，这可以提升编译器生成最优指令调度的能力 [4]。此外，Hopper架构支持通过 `setmaxnreg` 指令在Warp组之间动态重新分配寄存器 [40, §9.7.17.1]：使执行矩阵乘法 (MMA) 的warp拥有比只执行TMA数据加载的warp更多的寄存器 (后者每个CTA只需一个线程执行即可)。

低精度数值格式：现代GPU拥有专门的硬件单元加速低精度计算。例如，WGMMMA指令可针对Hopper架构上的FP8张量核心执行运算，相比FP16或BF16每个SM可提供2倍的吞吐量。然而，正确使用FP8 WGMMMA需要理解其操作数的存储布局限制。给定一个GEMM操作用于计算 $A \times B^T$ ，其中矩阵 A 尺寸为 $M \times K$ ，矩阵 B 尺寸为 $N \times K$ 。如果将矩阵展开为一维内存，我们称 A 或 B 是 **mn-major** 布局，表示在内存中按其外层维度 (M 或 N) 连续存储；称其为 **k-major** 布局则表示按内层的 K 维连续存储。对于FP16精度的WGMMMA，无论操作数以mn-major还是k-major格式存放于SMEM中都可被接受；但对于FP8精度的WGMMMA，操作数仅支持k-major格式。此外，在类似注意力这样的场景下，我们希望将两个矩阵乘法背靠背融合在同一个内核中，但FP32累加器和FP8操作数不同的布局要求会阻碍连续调用相关的FP8 WGMMMA指令。

在注意力计算背景下，以上布局限制意味着我们需对FP8算法设计进行一定改造，详见§3.3节。我们通过**分块量化**和**不相干处理**等技术减轻从FP16降至FP8精度导致的准确度下降。

2.3 标准注意力和FlashAttention

遵循Dao等人 [17] 的定义，我们将**标准注意力**指代在GPU上实现注意力的一种常规方法，该方法会将中间得分矩阵 S 和概率矩阵 P 完整地物化到HBM中。而FlashAttention的核心思想是在局部执行softmax归一化以避免这些高开销的中间读写，并将注意力计算融合为单个内核。所谓局部softmax，指的就是算法1中消费者主循环第18-19行的计算 (即对每个块进行softmax归一化) 以及对输出块 O 的重新缩放。可以参考文献 [15, §2.3.1] 找到这一过程确实能够得到正确 O 的简单推导。

3 FlashAttention-3：算法

在本节中，我们介绍FlashAttention-3算法。为简洁起见，我们首先聚焦前向计算过程（反向过程算法见附录B.1节）。我们首先说明如何将Warp特化和循环缓冲的共享内存（SMEM）机制集成进FlashAttention-2的基础算法中，随后解释如何利用异步执行隐藏softmax的开销，并最终讨论面向FP8低精度的特殊处理。

FlashAttention-2算法的基本流程如算法1所示。它采用双层循环：外层循环遍历 Q 的块（大小为 B_r ），内层循环遍历 K/V 的块（大小为 B_c ）。在每次内层循环迭代中，会计算当前 Q 块与一个 K 块的乘积生成得分块 $S_{i \wedge \{j\}} = Q_i K_j^T$ ，然后对该得分块进行softmax归一化，再与对应的 V 块相乘累加到输出块 O_i 上。为了避免超出FP32精度范围，每处理完一定数量的列块后，就对输出 O_i 进行重新缩放并累积偏移量 ℓ_i 。算法1中的ComputeSoftmax函数用于对一个得分块执行上述softmax和输出缩放。整个计算过程中，得分块 S 和概率块 P 等中间结果只在**共享内存**（SMEM）中保存，不会写回HBM，从而避免了大量全局内存访问。

算法1清晰地描述了FlashAttention-2的逻辑，但在实现上并未利用硬件的异步特性。我们的目标是对算法进行改造，使其能在保持数学等价的前提下，通过**Warp特化**和**流水线并行**来实现异步执行，以进一步提升性能。

Warp特化的循环缓冲区：在FlashAttention-2算法中，每次内层循环处理一个 K/V 块并立即计算对应的softmax和 O 累加。因此，计算和数据搬运是串行交替进行的。我们可以通过引入一个大小为 S 的**循环SMEM缓冲区**和Warp特化，将数据加载与计算的工作分配给不同的Warp组来实现重叠并行：专门的**生产者Warp组**负责将后续所需的数据块加载到SMEM缓冲区的下一位置上；**消费者Warp组**则在当前数据准备好后执行计算。当缓冲区满容量时，生产者需等待消费者处理腾出空间，而消费者也需等待下一个数据块载入后才能继续。这实际上形成了一种软件流水，在持续的内层循环迭代中，实现加载和计算交替的并行。

双阶段流水线：我们首先考虑一个两阶段的流水方案，在每次循环迭代中交替执行softmax和GEMM。具体而言，在消费者Warp组内部，我们将对当前得分块的softmax计算与下一得分块的矩阵乘法计算错开并行：在开始下一次WGMMMA矩阵乘法指令的同时，完成上一次softmax的剩余运算。这要求将softmax计算尽可能前移，并将WGMMMA调用尽可能后移，以制造二者的重叠窗口。算法2描述了这种两阶段流水的前向算法流程。我们用一个循环SMEM缓冲来暂存待处理的 Q 和相应的中间结果，每个缓冲槽对应一个流水阶段。

【算法2未显示为图表，用文字表述】算法2：FlashAttention-3 两阶段流水线（前向）

输入：查询、键、值矩阵 $Q, K, V \in \mathbb{R}^{N \times d}$ 存储在HBM中；block大小 B_c, B_r 。每个Warp组负责计算一个大小为 $B_r \times d$ 的输出子块 O_i 和对应的log-sum-exp向量 ℓ_i （大小为 B_r ）。共有 $T_c = \lceil N/B_c \rceil$ 个 K/V 子块（大小 $B_c \times d$ ）。

1. **初始化**：将当前 Q 子块 Q_i 从HBM加载到片上SRAM；初始化输出子块 O_i, ℓ_i, m_i, s_o 。
2. 等待生产者Warp组将第0号 K 子块 K_0 加载至片上SRAM。
3. 使用WGMMMA计算 $S = Q_i K_0^T$ （大小 $B_r \times B_c$ ）。提交（commit）并等待完成。
4. 基于 S 计算 $m_i, \tilde{P}_i, \ell_i, s_o$ 。（对 S 执行softmax得到 $\tilde{P}_i$ ，计算当前行最大值 m_i 、log-sum-exp值 ℓ_i ，计算输出缩放因子 s_o 等）
5. 等待生产者Warp组将第1号 K 子块 K_1 加载至片上SRAM。
6. 使用WGMMMA计算 $S = Q_i K_1^T$ 。提交并等待完成。
7. **循环开始** 对于 $j=2$ 到 T_c-2 ：
 - (a) 等待生产者Warp组将第 j 号 K 子块加载至SRAM；
 - (b) **并行阶段1**：用WGMMMA开始计算下一得分块 $S = Q_i K_j^T$ ，提交但不等待完成；
 - (c) **并行阶段2**：等待生产者Warp组将第 $j-2$ 号 V 子块加载至SRAM；根据前一次计算得到的 $\tilde{P}_i$ 将输出 O_i 按 s_o 重缩放；使用WGMMMA计算 $O_i \mathrel{+}= \tilde{P}_i V$ ，提交但不等待完成；

- (d) 计算 $m_i, \tilde{P}_{i, \text{next}}, \ell_i, s_o$ 基于刚得到的 SS ；
 - (e) 等待所有之前的WGMMA完成（确保阶段1、2的WGMMA都结束）；
 - (f) 将 SS 赋值为当前 SS ，将 $\tilde{P}_{i, \text{next}}$ 赋值为当前 $\tilde{P}_i$ ；（准备进入下一循环迭代）
8. 等待生产者Warp组将倒数第2号 V 子块 V 加载至SRAM；根据最后的 s_o 重缩放 O_i ；使用WGMMA计算 $O_i \mathrel{+} = \tilde{P}_i V$ ，提交并等待完成。
9. 基于最终 SS 计算 $m_i, \tilde{P}_i, \ell_i, s_o$ 作为结尾。

上述两阶段流水线算法的关键，是softmax和输出累积的计算与下一次块乘GEMM的执行互相重叠，从而减少空闲等待时间。通过硬件支持的Warp组同步指令（如 `WARGROUP.ARRIVE`、`WARGROUP.DEPBAR` 等）【40】和WGMMA指令，我们能够在指令级实现这种交错执行。我们在附录B.2中提供了一个Warp组消费者主循环内部的简化SASS（底层汇编）代码清单，以展示这种交错并行确实按预期工作：其中softmax的计算被重排到最前，第一组WGMMA指令与softmax和FP32→FP16数据转换指令穿插执行，第二组WGMMA指令按预期不与其它指令重叠。整体来看，底层SASS验证了我们的两阶段流水线方案的有效性。

三阶段流水线（尝试）：我们还尝试了一个三阶段的流水线算法，将“第 $j+2$ 次迭代的第一组WGMMA、 $j+1$ 次迭代的softmax以及第 j 次迭代的第二组WGMMA”三者并行化。算法4（见附录B.3）描述了这一方案。然而，由于额外的复杂性和调度冲突，这种三阶段流水线方案的效果不如两阶段方案，具体原因我们在附录B.3中讨论。简言之，三阶段方案并未显著胜过两阶段方案，因此我们的最终实现基于两阶段流水线。

3.1 Warp特化的异步执行（前向）

生产者-消费者模式：为了利用前述异步执行能力，我们将一个CTA中的warp划分为不同角色，部分warp专职执行数据加载（生产者），其余warp专职执行计算（消费者）。通过这种Warp特化，异步的TMA内存加载和张量核心计算可以重叠，最大化GPU硬件利用率。具体来说，在我们的实现中，每个CTA包含一个生产者Warp组和若干消费者Warp组。生产者Warp组负责将 K 和 V 子块从HBM加载到共享内存；而消费者Warp组在数据准备好后立即执行矩阵乘法和softmax计算。

循环缓冲区：我们为每个CTA分配一个大小为 s 的循环共享内存缓冲区，如算法2所示，用于暂存即将被处理的 Q 、 V 子块以及计算得到的中间值。该缓冲区通过分段分配确保生产者和消费者Warp组可以并发访问不同的缓冲区槽。借助缓冲区，我们实现了一个软件流水线：生产者Warp组连续预取后续需要的 Q 、 K 、 V 子块到缓冲区，而消费者Warp组从缓冲区读取已准备的数据块进行计算。当缓冲区满时，生产者等待消费者处理掉旧的数据；当缓冲区空时，消费者等待新的数据载入——从而在整体上实现数据搬运与计算的重叠。

结果：通过Warp特化和循环缓冲，我们扩展了FlashAttention-2算法的并行能力。消费者Warp组无需等待每次加载完成就能开始部分计算，充分利用了Hopper架构上张量核心（WGMMA）和内存加载（TMA）的异步特性，使GEMM和softmax阶段交替重叠执行。这极大减少了计算单元的等待时间，提高了硬件利用率。我们在后文的实验中将展示，这种异步流水线策略对性能提升的贡献是非常显著的。

3.2 在异步GEMM下隐藏Softmax开销

上一节介绍的两阶段流水线方案是我们异步算法设计的核心，它实现了块乘GEMM与softmax计算的交错执行。在实现该方案时，有若干具体的挑战需要克服：

- **softmax提前计算**：为了在第一组WGMMA执行时就并行进行softmax计算，我们需将softmax的部分计算尽可能前移。然而，在标准的注意力算法中，softmax的计算依赖完整的得分块 SS 结果。因此，我们需要调整算法顺序，使得在得到 SS 部分结果时就能开始softmax的相关计算。

- **结果累积与重新缩放**：注意力计算会在输出\$O\$上累积多个部分结果，并定期进行重新缩放防止数值溢出。在流水线中，我们必须确保这些累积和缩放操作与WGMMMA的执行妥善交错，并且不破坏计算的正确性。
- **指令调度与同步**：为了让上述并行顺利进行，我们依赖Hopper架构新增的Warp组级同步指令（如 `WARGROUP.ARRIVE` 通知另一Warp组数据已准备好，`WARGROUP.DEPBAR` 等待另一Warp组完成计算）。我们需要精心插入这些同步指令，配合TMA和WGMMMA指令，使得内存加载、GEMM和softmax三个过程准确同步又互不阻塞。

通过解决这些问题，我们实现了算法2描述的双阶段流水方案。在附录B.2中，我们提供的SASS片段显示：在实际底层执行中，softmax计算已经被提前到第一个WGMMMA之前开始，WGMMMA指令与softmax和数据类型转换指令交错执行，这证明了我们成功地将softmax的开销隐藏在异步GEMM执行之下。

3.3 低精度（FP8）计算的特别处理

将精度降低到FP8可以极大提升计算吞吐量，但同时也会引入额外的挑战。正如前文讨论的，Hopper架构FP8 WGMMMA对操作数内存布局的要求更加严格。此外，FP8的数值动态范围有限，在训练中直接使用FP8常会导致不稳定【21】。在FlashAttention-3中，我们使用以下两种技术来应对FP8的精度问题：

1. **分块量化**：不像逐个张量使用一个量化比例因子，我们对每个矩阵按照小块划分，每块使用独立的缩放因子进行FP8量化。具体来说，我们将\$Q, K, V\$每个张量划分为若干块，每块都有自己的量化尺度。这减少了由于单个全局尺度不能同时兼顾巨大小值而导致的量化误差。
2. **不相干处理**：为了进一步降低量化误差，我们采用类似QuIP【9】和QuIP#【58】的方法，打乱（使不相干）FP8量化过程，避免系统性的量化偏差累积。这包括对激活值分布进行预处理，使得量化误差更像随机噪声而非有偏的系统误差。

我们在§4.3节验证了以上两种技术可以使FP8精度下的注意力计算误差降低最多2.6倍。

在FP8 WGMMMA调用方面，我们调整算法以满足其布局要求。例如，对于FP8，我们确保\$K\$矩阵块在内存中按\$K\$维（列）连续，以符合FP8 WGMMMA的k-major要求；在融合多个GEMM时，我们对累加器布局进行转换以消除与FP8操作数布局的不兼容。这些工程细节超出了本文篇幅，可参考我们开源代码和附录中的讨论。

综上，通过Warp特化的异步流水和对FP8的专门处理，FlashAttention-3充分利用了Hopper架构的新硬件特性，大幅提升了注意力计算的速度，同时兼顾计算精度。下一节我们将展示实验证据。

4 实验验证

4.1 注意力性能基准测试

我们在一块H100 80GB SXM5 GPU（700W功耗）上对不同注意力计算方法的运行时间进行了测量，对比了有无因果掩码、head维度为64或128等不同配置，并统一使用FP16输入。我们报告前向和后向传播的平均运行时间（或等效的计算吞吐量）。对比方法包括FlashAttention-2、FlashAttention-3以及NVIDIA cuDNN库中提供的高性能注意力实现等。

结果显示，在FP16精度下，无论有无因果掩码，FlashAttention-3相比FlashAttention-2都取得了明显加速：前向计算提速约1.5–2倍，后向计算提速约1.5倍以上。而与NVIDIA cuDNN 9.1实现相比，FlashAttention-3在长序列（如序列长度大于8k）下前向计算性能持平甚至略有超出，在短序列下后向计算性能也有明显优势（短序列情况下可超过cuDNN约40%，长序列下约10%的优势）。

上述性能提升得益于我们提出的流水线并行和Warp特化技术。在相同硬件上，我们的实现达到的峰值前向吞吐量约为740 TFLOP/s（75%理论性能），相比之下FlashAttention-2仅约400 TFLOP/s，证明了我们方法显著更高的硬件利用效率。【¹⁰】

4.2 消融实验：两阶段流水线

我们对“两阶段WGMMMA-softmax流水线”和“Warp特化”两个关键策略进行了消融实验。在仅FP16、关闭因果掩码的配置下（batch=4，序列长度=8448，注意力头数=16，head维度=128），我们对比了以下三种实现变体的性能：(a) 同时启用两阶段流水线和Warp特化（即完整的FlashAttention-3）；(b) 仅启用Warp特化但不执行softmax/GEMM交错（相当于各warp角色固定，但按顺序完成各步骤）；(c) 不启用Warp特化，使用传统的串行实现。结果表明，同时启用两阶段流水线和Warp特化的完整方案性能最佳；移除Warp特化（方案c）会导致性能下降约30%；而仅移除softmax/GEMM交错（方案b）性能也有显著下降。这证明了我们提出的两项技术对性能提升都是不可或缺的。

4.3 数值误差验证

近期有研究关注FlashAttention的数值稳定性问题【21】。在本节中，我们比较了FlashAttention-2、FlashAttention-3和标准注意力在数值精度方面的差异。我们以高精度（例如FP64或模拟无限精度）计算结果作为参考基准，计算上述各实现在前向计算输出上的均方根误差（RMSE）。在FP16精度下，FlashAttention-2与FlashAttention-3的误差基本相当，均保持在 10^{-4} 量级，显著优于标准注意力实现（标准实现由于中间结果截断，误差约 3.2×10^{-4} ，而FlashAttention保持在 1.9×10^{-4} ）。在FP8精度（采用e4m3格式）下，标准注意力由于逐张量量化造成的误差非常大（约 2.4×10^{-2} ）；引入分块量化的FlashAttention-2将误差降低一个量级（约 9×10^{-3} ）；而FlashAttention-3进一步通过不相干处理将误差降至约 9.3×10^{-3} ，相当于比标准FP8注意力误差降低了约2.6倍。

需要强调的是，虽然FP8精度在训练中尚未普遍使用，但我们的结果表明，通过适当的策略（如分块量化和不相干处理），FP8在注意力计算中是可行且精度可控的。这为将来的研究（例如在保持训练稳定性的情况下使用更低精度计算）提供了参考方向。

[⁴]: Luo等人【34】报告Hopper架构中每个SM的共享内存带宽约为每周周期128字节。乘以132个SM和1.83 GHz的SM时钟频率，可得总带宽约31 TB/s。

5 讨论、局限与结论

借助FlashAttention-3，我们证明了新的编程技术和硬件特性（如异步执行和低数值精度）能够对注意力计算的效率产生巨大的影响。在Transformer等模型中，注意力层长期以来一直是性能瓶颈，而我们的工作表明，通过深入理解并利用硬件的新能力，可以大幅缓解这一瓶颈。我们的实现显著提高了H100 GPU上的注意力吞吐率，使其更接近硬件峰值。

讨论与局限性：我们的工作主要针对训练阶段的注意力计算进行了优化。在推理阶段，由于长序列主要受KV缓存读写的影响，我们的优化收益可能有所不同。目前的实现尚未针对大语言模型（LLM）的推理进行特定优化，这将是我们的未来工作之一。此外，我们计划探索将持久化kernel机制引入FP8精度的实现，以进一步提升其在小序列和有因果场景下的性能。

另一个值得注意的问题是硬件的兼容性。我们的设计充分利用了Hopper架构的新特性，因而在老一代GPU（如Ampere架构）上无法获得同等收益。随着新的Blackwell架构推出FP4精度，我们的方法也需要相应调整以利用FP4等更低精度。如何在保持训练稳定性的同时利用这些更低精度将是一个挑战。

结论：本研究表明，通过软硬件协同设计，可以大幅提升Transformer注意力机制的计算性能。FlashAttention-3利用了Warp特化异步执行和低精度计算，将H100 GPU的注意力运算性能提升至前所未有的水平。在大型模型和长上下文的趋势下，我们的工作为高效训练和推理提供了新的可能。我们相信，这些加速注意力的方法同样适用于一些替代架构（如结合SSM或RNN的混合模型）中的注意力模块，加速未来的大模型训练与推理。

参考文献

- [1] Ahmad Abdelfattah, Azzam Haidar, Stanimire Tomov, and Jack Dongarra. Performance, design, and autotuning of batched GEMM for GPUs. pp. 21–38, 06 2016. ISBN 978-3-319-41320-4. doi: 10.1007/978-3-319-41321-1_2.
- [2] AI21. Introducing Jamba: AI21’s groundbreaking SSM-Transformer model. AI21 Blog, 2024.
- [3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query Transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.
- [4] Michael Bauer, Henry Cook, and Brucek Khailany. CudaDMA: Optimizing GPU Memory Bandwidth via Warp Specialization. In *Proceedings of SC ’11: International Conference for High Performance Computing, Networking, Storage and Analysis*, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450307710. doi: 10.1145/2063384.2063400. URL: <https://doi.org/10.1145/2063384.2063400>.
- [5] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xLSTM: Extended long short-term memory. arXiv preprint arXiv:2405.04517, 2024.
- [6] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document Transformer. arXiv preprint arXiv:2004.05150, 2020.
- [7] Ganesh Bikshandi and Jay Shah. Delivering 1 PFLOP/s of Performance with FP8 FlashAttention-2, 2024. URL: <https://research.colfax-intl.com/adding-fp8-to-flashattention/>.
- [8] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. Striped attention: Faster ring attention for causal Transformers. arXiv preprint arXiv: 2311.09431, 2023.

- [9] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M. De Sa. QuIP: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems*, 36, 2024.
- [10] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [11] Richard J. Chen, Chengkuan Chen, Yicong Li, Tiffany Y. Chen, Andrew D. Trister, Rahul G. Krishnan, and Faisal Mahmood. Scaling vision Transformers to gigapixel images via hierarchical self-supervised learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16144–16155, 2022.
- [12] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse Transformers. arXiv preprint arXiv:1904.10509, 2019.
- [13] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Łukasz Kaiser, *et al.* Rethinking attention with Performers. In *International Conference on Learning Representations (ICLR)*, 2021.
- [14] Krzysztof Marcin Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Łukasz Kaiser, *et al.* Rethinking attention with Performers. In *International Conference on Learning Representations (ICLR)*, 2020.
- [15] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, 2023. URL: <https://arxiv.org/abs/2307.08691>.
- [16] Tri Dao and Albert Gu. Transformers are SSMS: Generalized models and efficient algorithms with structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024.
- [17] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [18] Tri Dao, Daniel Y. Fu, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [19] DeepSeek-AI. DeepSeek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434, 2024.
- [20] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for Transformers at scale. CoRR abs/2208.07339, 2022.
- [21] Alicia Golden, Samuel Hsia, Fei Sun, Bilge Acun, Basil Hosmer, Yejin Lee, Zachary DeVito, Jeff Johnson, Gu-Yeon Wei, David Brooks, *et al.* Is FlashAttention stable? arXiv preprint arXiv:2405.02803, 2024.
- [22] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023.

- [23] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, *et al.* Conformer: Convolution-augmented Transformer for speech recognition. arXiv preprint arXiv:2005.08100, 2020.
- [24] Mandy Guo, Joshua Ainslie, David Uthus, Santiago Ontañón, Jianmo Ni, Yun-Hsuan Sung, and Yinfei Yang. LongT5: Efficient text-to-text Transformer for long sequences. arXiv preprint arXiv:2112.07916, 2021.
- [25] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J. Fleet. Video diffusion models. *Advances in Neural Information Processing Systems*, 35:8633–8646, 2022.
- [26] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQant: Towards 10 million context length LLM inference with KV cache quantization. arXiv preprint arXiv:2401.18079, 2024.
- [27] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive Transformers with linear attention. In *International Conference on Machine Learning (ICML)*, pp. 5156–5165. PMLR, 2020.
- [28] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient Transformer. In *International Conference on Machine Learning (ICML)*, 2020.
- [29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023.
- [30] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, *et al.* StarCoder: May the source be with you! arXiv preprint arXiv:2305.06161, 2023.
- [31] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise Transformers for near-infinite context. arXiv preprint arXiv:2310.01889, 2023.
- [32] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with RingAttention. arXiv preprint arXiv:2402.08268, 2024.
- [33] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2-bit quantization for KV cache. arXiv preprint arXiv:2402.02750, 2024.
- [34] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Qiang Wang, and Xiaowen Chu. Benchmarking and dissecting the NVIDIA Hopper GPU architecture, 2024. URL: <https://arxiv.org/abs/2402.13499>.
- [35] Xuezhe Ma, Chunting Zhou, Xiang Kong, Junxian He, Liangke Gui, Graham Neubig, Jonathan May, and Luke Zettlemoyer. MEGA: Moving average equipped gated attention. In *International Conference on Learning Representations (ICLR)*, 2023.

- [36] Xuezhe Ma, Xiaomeng Yang, Wenhan Xiong, Beidi Chen, Lili Yu, Hao Zhang, Jonathan May, Luke Zettlemoyer, Omer Levy, and Chunting Zhou. Megalodon: Efficient LLM pretraining and inference with unlimited context length. arXiv preprint arXiv:2404.08801, 2024.
- [37] Paulius Micikevicius, Dušan Stojić, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, *et al.* FP8 formats for deep learning. arXiv preprint arXiv:2209.05433, 2022.
- [38] NVIDIA. *CUDA C Programming Guide*, Version 12.4, 2024. URL: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [39] NVIDIA. Accelerating Transformers with NVIDIA cuDNN 9. NVIDIA Blog, 2024. URL: <https://developer.nvidia.com/blog/accelerating-transformers-with-nvidia-cudnn-9/>.
- [40] NVIDIA. *Parallel Thread Execution ISA*, Version 8.4, 2024. URL: https://docs.nvidia.com/cuda/pdf/ptx_isa_8.4.pdf.
- [41] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. Stream-K: Work-centric parallel decomposition for dense matrix-matrix multiplication on the GPU. In *Proceedings of PPoPP '23: 28th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 429–431, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700156. doi: 10.1145/3572848.3577479.
- [42] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, *et al.* RWKV: Reinventing RNNs for the Transformer era. arXiv preprint arXiv:2305.13048, 2023.
- [43] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YARN: Efficient context window extension of large language models. arXiv preprint arXiv:2309.00071, 2023.
- [44] Hao Peng, Nikolaos Pappas, Dani Yogatama, Roy Schwartz, Noah A. Smith, and Lingpeng Kong. Random feature attention. In *International Conference on Learning Representations (ICLR)*, 2021.
- [45] Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. arXiv preprint arXiv:2112.05682, 2021.
- [46] Colfax Research. Tutorial: Matrix Transpose in CUTLASS, 2024. URL: <https://research.colfax-intl.com/tutorial-matrix-transpose-in-cutlass/>.
- [47] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing Transformers. arXiv preprint arXiv:2003.05997, 2020.
- [48] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, *et al.* Code Llama: Open foundation models for code. arXiv preprint arXiv:2308.12950, 2023.

- [49] Rya Sanovar, Srikant Bharadwaj, Renee St. Amant, Victor Rühle, and Saravan Rajmohan. Lean attention: Hardware-aware scalable attention mechanism for the decode-phase of Transformers, 2024.
- [50] Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, *et al.* SCROLLS: Standardized comparison over long language sequences. arXiv preprint arXiv:2201.03533, 2022.
- [51] Noam Shazeer. Fast Transformer decoding: One write-head is all you need. arXiv preprint arXiv:1911.02150, 2019.
- [52] Benjamin Spector, Aaryan Singhal, Simran Arora, and Christopher Ré. ThunderKittens (code library), 2024. URL: <https://github.com/HazyResearch/ThunderKittens>.
- [53] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. BERT4Rec: Sequential recommendation with bidirectional encoder representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, pp. 1441–1450, 2019.
- [54] Mingjie Sun, Xinlei Chen, J. Zico Kolter, and Zhuang Liu. Massive activations in large language models. arXiv preprint arXiv:2402.17762, 2024.
- [55] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to Transformer for large language models. arXiv preprint arXiv:2307.08621, 2023.
- [56] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient Transformers: A survey. arXiv preprint arXiv:2009.06732, 2020.
- [57] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS, January 2023. URL: <https://github.com/NVIDIA/cutlass>.
- [58] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. Quip#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. arXiv preprint arXiv:2402.04396, 2024.
- [59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [60] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, *et al.* An empirical study of Mamba-based language models. arXiv preprint arXiv:2406.07887, 2024.
- [61] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A Nyström-based algorithm for approximating self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, p. 14138, 2021.

[62] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629, 2022.

[63] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, *et al.* Big Bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

[64] Zyphra. Zyphra unveils Zamba: A compact 7B SSM hybrid model. Zyphra Blog, 2024.
